



COMPILADORES 2025-1

Profesora: Ariel Adara Mercado Martínez

Ayudante: Carlos Gerardo Acosta Hernández

Práctica 4: Analizadores sintácticos con BYACC/J (YACC).

Nombre	No. de Cuenta
Vázquez Torrijos Damián	318309877

21 de Octubre del 2024

Para comodidad, se abreviaron las producciones:

- programa = Pr
- declaraciones = Ds
- declaracion = D
- tipo = T
- lista_var = L_V
- sentencias = Ss
- sentencia = S
- expresion = E

1. Determinar los conjuntos N , Σ y el símbolo inicial S .

- $N = \{Pr, Ds, D, Ss, S, T, L_V, E\}$
- $\Sigma = \{;, id, =, if, else, while, +, -, *, /, (,), num\}$
- $S = \{Pr\}$

2. Mostrar el proceso de eliminación de ambigüedad o justificar, en caso de no ser necesario.

La producción E tiene ambigüedad, para eliminarla debemos definir el orden de procedencia de los operadores de menor a mayor:

- $+$ $-$
- $*$ $/$
- $()$

Una vez definido el orden, hacemos nuevas producciones en base al algoritmo de la eliminación de ambigüedad:

- $E \rightarrow E + F | E - F | F$
- $F \rightarrow F * G | F / G | G$
- $G \rightarrow (E) | id | num$

3. Mostrar el proceso de eliminación de la recursividad izquierda o justificar, en caso de no ser necesario.

Las producciones Ds, L_V, Ss, E, F tienen recursividad izquierda, por lo que hacemos nuevas producciones en base al algoritmo de la eliminación de la recursividad izquierda:

- $Ds \rightarrow DDs'$
 $Ds' \rightarrow DDs'|\epsilon$
- $L_V \rightarrow idL_V'$
 $L_V' \rightarrow, idL_V'|\epsilon$
- $Ss \rightarrow SSs'$
 $Ss' \rightarrow SSs'|\epsilon$
- $E \rightarrow FE'$
 $E' \rightarrow +FE'|-FE'|\epsilon$
- $F \rightarrow GF'$
 $F' \rightarrow *GF'|/GF'|\epsilon$

4. Mostrar el proceso de factorización izquierda o justificar, en caso de no ser necesario.

No hay producciones que se puedan factorizar debido a que no son de la forma $A \rightarrow \alpha\beta_1|\alpha\beta_2|\alpha\beta_3|\dots|\alpha\beta_n|\gamma_1|\gamma_2|\dots|\gamma_n$, es decir, no tienen prefijo en común.

5. Mostrar los nuevos conjuntos N y P .

- $N = \{Pr, Ds, Ds', D, T, L_V, L_V', Ss, Ss', S, E, E', F, F', G, G', H, H', I\}$
- $P = \{Pr \rightarrow DsSs$
 $Ds \rightarrow DDs'$
 $Ds' \rightarrow DDs'|\epsilon$
 $D \rightarrow TL_V;$
 $T \rightarrow int|float$
 $L_V \rightarrow idL_V'$
 $L_V' \rightarrow, idL_V'|\epsilon$
 $Ss \rightarrow SSs'$
 $Ss' \rightarrow SSs'|\epsilon$
 $S \rightarrow id = E; |if(E)SselseSs|while(E)Ss$
 $E \rightarrow FE'$
 $E' \rightarrow +FE'|-FE'|\epsilon$
 $F \rightarrow GF'$
 $F' \rightarrow *GF'|/GF'|\epsilon$
 $G \rightarrow (E)|id|num\}$

6. Realizar cualquier otro tratamiento necesario para evitar conflictos de shift/reduce mostrando el proceso.

7. Crear una definición con BYACC/J para la gramática resultante.

```

/* Seccion de declaraciones*/
...
%%

pr : ds ss
;

ds : d dsp
;

dsp : d dsp
|
;

d : t lv PYC
;

t : INT
| FLOAT
;

lv : ID lvp
;

lvp : COMA ID lvp
|
;

ss : s ssp
;

ssp : s ssp
|
;

s : ID ASIG e PYC
| IF LPAR e RPAR ss ELSE ss
| WHILE LPAR e RPAR ss

```

```

;

e : f ep
;

ep : SUMA f ep
   | RESTA f ep
   |
;

f : g fp
;

fp : MULT g fp
   | DIV g fp
   |
;

g : LPAR e RPAR
   | ID
   | NUM
;

%%
/* Seccion de codigo */
...

```